

Trabalho Final Seg Info

Diego V. S. de Matos – 120098723

December 6, 2025

1 Protocolo TLS

O TLS é projetado para usar o TCP para prover um serviço seguro, é caracterizado por ter duas camadas de protocolos como descrito abaixo.

O **Protocolo de Registro** (*Record Protocol*) fornece os serviços básicos de segurança para os protocolos da camada superior:

- Confidencialidade: via criptografia simétrica
- Integridade: via hash criptográfico
- Autenticação: via criptografia de chave pública

Os **Protocolos Superiores** (**Gerenciamento TLS**) é responsável pelo Handshake, que é definido em 4 passos:

1. Passo 1: Inicializa conexão lógica e estabelece as capacidades de segurança. Essa troca é inicializada pelo cliente
 - cliente envia *client_hello*
 - cliente envia a versão do TLS mais alta compatível
 - uma sequência inicial aleatória de bytes, usado para criptografia e para evitar ataques de replay
 - ID de sessão
 - indica conjuntos de cifras que suportado
 - Lista de métodos de compressão suportados
 - aguarda a resposta *server_hello* do servidor:
 - servidor responde com os mesmos parâmetros do *client_hello*
 - a cifra escolhida
 - o método de compressão escolhido
2. Passo 2: Opcionalmente servidor envia o seu certificado para o cliente autenticar e informações da chave (*ServerKeyExchange*). Essa etapa termina obrigatoriamente com *server_done*

- Opcionalmente o servidor solicita um certificado do cliente (*CertificateRequest*)
3. Passo 3: Ao receber *server_done*, o cliente irá verificar, caso necessário, se o certificado do servidor foi emitido por uma autoridade de certificação confiável, se é válido e não revogado e se o domínio é o mesmo. Caso tudo estiver, tudo certo o cliente responde ao servidor.
 4. Finalização: O cliente envia uma mensagem de *change_cipher_spec*, que sinaliza que a partir deste momento as mensagens estarão criptografadas, e imediatamente envia mensagem *finished* que verifica se a troca de chaves e a autenticação foram bem-sucedidas. O servidor também responde com *change_cipher_spec* e com sua própria mensagem de *finished*.
O handshake está completo e o cliente e servidor podem começara trocar dados da camada de aplicação de forma segura.

2 Comparação de Implementações

Foi implementado em Python dois programas cliente e servidor para cada caso: com TLS e sem TLS. As rotinas para cada caso são parecidas e suas diferenças principais são:

- 1) Inicialização dos programas: No caso com TLS o servidor deve carregar os certificados e criar seu socket normalmente

Listing 1: Inicializacao do Servidor

```
# Rotina abaixo apenas para o caso com TLS
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile="cert.pem", keyfile="key.pem")

# Rotina abaixo em comum para ambos os casos
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
```

Como a aplicação está rodando em localhost, para praticidade o cliente não irá checar o certificado do servidor. Isso não influencia na criptografia TLS.

Listing 2: Inicializacao do Cliente

```
# Apenas para o caso com TLS
context = ssl.create_default_context()
context.check_hostname = False
# Para nao verificar o certificado
context.verify_mode = ssl.CERT_NONE
```

- 2) Em Python, ao importar o pacote *ssl*, temos acesso a uma API para fazer um envólucro na conexão do socket, dessa forma facilitando o envio da mensagem da aplicação pois é feita a encriptação e deciptação de forma automática tanto no cliente como no servidor. Passo não necessário no caso sem TLS.

Listing 3: Conexão do servidor

```
conn, addr = server.accept()
# Conexão segura com TLS usando envelope python
secure_conn = context.wrap_socket(conn, server_side=True)
```

Listing 4: Conexão do cliente

```
raw_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Conexão segura com TLS usando envelope python
secure_sock = context.wrap_socket(raw_sock, server_hostname=HOST)
```

3 Captura dos Pacotes

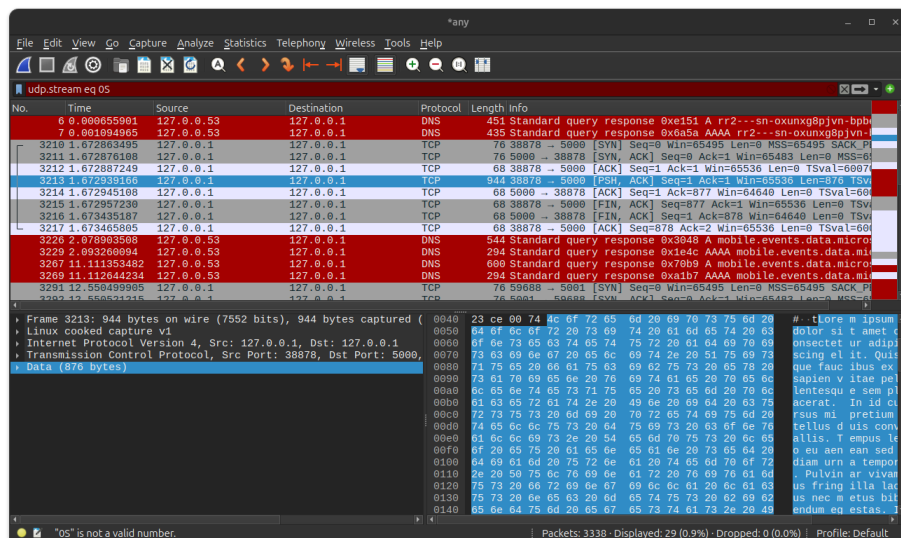


Figure 1: Pacotes enviados sem tls

Na Figura 1 conseguimos analisar os pacotes para o caso sem TLS. Observamos uma sequência de pacotes padrão do protocolo TCP (SYN, ACK) entre o cliente e servidor em localhost (127.0.0.1) nas portas 38879 → 5000 seguido imediatamente do conteúdo da aplicação e está em *plain text*.

Já para o caso com TLS observamos na Figura 2 a conexão entre o cliente e servidor em localhost (127.0.0.1) nas portas 59699 → 5001 respectivamente. Os três primeiros pacotes são referentes ao TCP, o que já era esperado pois o TLS é executado em cima do TCP. É seguido dos pacotes de *Client Hello*, *Server Hello* e *Change Cipher Spec*, mostrando que de fato o handshake do TLS funcionou como esperado. Como o meu programa está rodando em localhost estou pulando

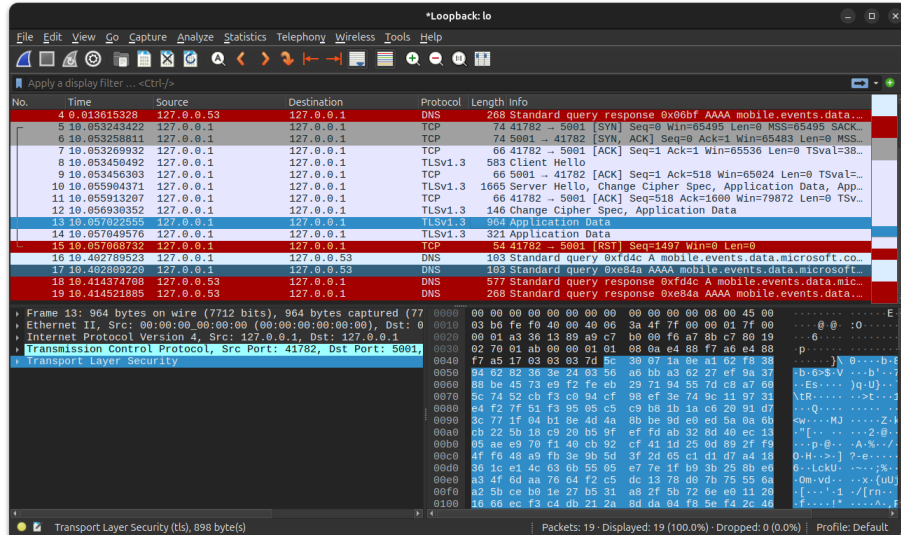


Figure 2: Pacotes enviados com tls

o passo de verificar o certificado, dessa forma o próximo pacote já é o conteúdo cifrado. Esse fluxo é finalizado com o cliente enviando pacote de RST (RESET), que encerra a conexão.

4 Análise de overhead

Usando o wireshark no menu Statistics → Conversations → TCP, foram coletados dados das conexões com e sem TLS, portas 5001 e 5000 respectivamente, que foram salvos no arquivo dados.csv. Nessa tabela vemos:

- na conexão com TLS foram enviados 3 pacotes adicionais
- O total de bytes trafegados com TLS foi de 4079 bytes, contra 1436 bytes sem TLS, um aumento de aproximadamente 284%.
- O tempo total da conexão com TLS foi de $38.2531 \times 10^{-4}s$, enquanto sem TLS foi de $6.086120 \times 10^{-4}s$. A diferença foi de cerca de $32.16698 \times 10^{-4}s$.

A utilização de TLS introduz necessariamente algum overhead, tanto em termos de quantidade de pacotes quanto de tempo de processamento e latência. Esse custo decorre principalmente de dois fatores:

1. **Custo do Handshake:** o handshake emprega criptografia assimétrica, que é computacionalmente mais pesada que a simétrica. O envio de mensagens adicionais (ClientHello, ServerHello, troca de chaves e ChangeCipherSpec) aumenta o número de pacotes antes que a transmissão de dados da aplicação possa começar.

Em conexões curtas, como no cenário deste experimento, o handshake representa a maior parte do custo total.

2. **Cifragens nos Dados de Aplicação:** embora a criptografia simétrica seja eficiente, ainda há um custo adicional para cifrar e decifrar cada bloco de dados transmitido. Esse custo é pequeno em máquinas modernas, mas existe.

Outro fator relevante é que a análise foi realizada em *localhost*. Nesse ambiente:

- A latência de rede é praticamente nula.
- O custo da criptografia domina a análise.
- O impacto do handshake torna-se proporcionalmente maior, pois a transmissão de dados é muito rápida.

Em redes reais (WAN, Wi-Fi, enlaces de longa distância), o comportamento pode ser diferente:

- O número extra de pacotes do TLS tem impacto maior devido ao tempo de ida e volta (RTT).
- Em conexões persistentes (HTTP/2, long-lived connections), o custo do handshake pode ser amortizado ao longo de muitas mensagens.
- Implementações otimizadas de TLS (como TLS 1.3) reduzem substancialmente o número de mensagens do handshake e melhoram a performance.

Embora o TLS adicione overhead perceptível em conexões muito curtas, especialmente em ambientes locais, ele é perfeitamente aceitável em situações reais, nas quais a segurança dos dados é imprescindível. Para a maioria dos sistemas modernos, o impacto adicional é pequeno diante dos benefícios proporcionados.